

게임 수학

강의 8: 행렬과 변환

좌표계, 어파인 변환, 동차 좌표계, 회전 행렬

동명대학교 게임공학과
강영민

2026년 1학기

Contents

1	좌표계	2
2	선형 변환과 어파인 변환	2
3	동차 좌표계 (Homogeneous Coordinates)	3
4	기본 변환 행렬	3
4.1	이동 (Translation)	3
4.2	스케일링 (Scaling)	4
4.3	2D 회전	4
4.4	3D 회전	4
5	변환의 합성	5
6	오일러 각도 (Euler Angles)	6
7	역변환과 법선 벡터 변환	6
7.1	역변환	6
7.2	법선 벡터 변환	6
8	뷰 행렬과 투영 행렬	7
8.1	뷰 행렬 (View Matrix)	7
8.2	원근 투영 행렬 (Perspective Projection)	7
9	Python 구현	7
10	연습 문제	10
11	요약	10

1 좌표계

3D 그래픽에서는 여러 좌표계(coordinate system)가 동시에 사용된다. 변환 행렬은 한 좌표계에서 다른 좌표계로 점/벡터를 옮기는 역할을 한다.

좌표계 종류

- 직교 좌표계(Cartesian): (x, y, z) — 가장 일반적
- 원통 좌표계(Cylindrical): (r, ϕ, z) — 원통형 대칭
- 구면 좌표계(Spherical): (r, θ, ϕ) — 구면 대칭

게임 파이프라인에서의 좌표계.

1. 오브젝트 공간(Object/Local Space): 오브젝트 중심 기준
2. 월드 공간(World Space): 씬(scene) 전체 기준
3. 뷰 공간(View/Camera Space): 카메라 기준
4. 클립 공간(Clip Space): 투영(projection) 후
5. 화면 공간(Screen Space): 픽셀 좌표

각 단계 사이의 변환은 행렬로 표현된다: 오브젝트 공간 $\xrightarrow{M}$ 월드 $\xrightarrow{V}$ 뷰 $\xrightarrow{P}$ 클립 $\rightarrow$ 화면

2 선형 변환과 어파인 변환

정의 — 선형 변환 (Linear Transformation)

함수 $T: \mathbb{R}^n \rightarrow \mathbb{R}^m$ 이 다음을 만족하면 선형 변환이다:

1. $T(\mathbf{u} + \mathbf{v}) = T(\mathbf{u}) + T(\mathbf{v})$ (가산성)
2. $T(k\mathbf{v}) = kT(\mathbf{v})$ (동차성)

이는 $T(\mathbf{0}) = \mathbf{0}$ (원점 보존)을 내포한다.

선형 변환의 예: 회전, 스케일링, 반사, 전단(shear).

정의 — 어파인 변환 (Affine Transformation)

어파인 변환은 선형 변환 + 이동(translation)이다:

$$T(\mathbf{x}) = A\mathbf{x} + \mathbf{t}$$

여기서 A 는 선형 변환 행렬, $\mathbf{t}$ 는 이동 벡터.

왜 이동이 선형 변환이 아닌가?

$T(\mathbf{x}) = \mathbf{x} + \mathbf{t}$ 라 하면, $T(\mathbf{0}) = \mathbf{t} \neq \mathbf{0}$ 이므로 선형 변환의 정의를 만족하지 않는다. 따라서 이동을 순수한 $n \times n$ 행렬 곱으로는 표현할 수 없다. 이 문제를 해결하는 것이 동차 좌표계이다.

3 동차 좌표계 (Homogeneous Coordinates)

정의 — 동차 좌표계

n 차원 점/벡터에 동차 성분 w 를 추가하여 $(n+1)$ 차원으로 확장:

- 점(point): $(x, y, z, 1)^T$ — $w = 1$
- 방향 벡터(direction vector): $(x, y, z, 0)^T$ — $w = 0$

동차 좌표 (x, y, z, w) 는 3D 직교 좌표 $(x/w, y/w, z/w)$ 에 해당한다 ($w \neq 0$).

$w = 1$ vs $w = 0$ 의 의미.

- $w = 1$ (점): 이동 변환의 영향을 받는다.
- $w = 0$ (방향): 이동 변환의 영향을 받지 않는다. 법선 벡터, 속도 벡터 등은 위치와 무관하게 방향만 있으므로 $w = 0$.

4 기본 변환 행렬

4.1 이동 (Translation)

이동 행렬 (Translation Matrix)

(t_x, t_y, t_z) 만큼 이동하는 4×4 동차 변환 행렬:

$$T(t_x, t_y, t_z) = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

적용: $(x, y, z, 1)^T$ 에 곱하면 $(x + t_x, y + t_y, z + t_z, 1)^T$.

방향 벡터 $(x, y, z, 0)^T$ 에 곱하면 $(x, y, z, 0)^T$ (불변). ✓

역행렬: $T^{-1}(t_x, t_y, t_z) = T(-t_x, -t_y, -t_z)$.

4.2 스케일링 (Scaling)

스케일 행렬 (Scale Matrix)

각 축 방향으로 s_x, s_y, s_z 배 확대/축소:

$$S(s_x, s_y, s_z) = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

균등 스케일 (uniform scale): $s_x = s_y = s_z = s$.

역행렬: $S^{-1}(s_x, s_y, s_z) = S(1/s_x, 1/s_y, 1/s_z)$.

4.3 2D 회전

2D 회전 행렬

원점을 중심으로 각도 θ 만큼 반시계 회전 (행 벡터 기준):

$$R(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$$

2D 회전 행렬 유도. x 축 단위벡터 $(1, 0)$ 이 $(\cos \theta, \sin \theta)$ 으로 변환되고, y 축 단위벡터 $(0, 1)$ 이 $(-\sin \theta, \cos \theta)$ 으로 변환되어야 한다. 이 두 열을 붙이면 위 행렬이 된다.

또는 각도 덧셈 공식에서 유도:

$$x' = x \cos \theta - y \sin \theta$$

$$y' = x \sin \theta + y \cos \theta$$

역행렬: $R^{-1}(\theta) = R(-\theta) = R^T(\theta)$ (직교 행렬이므로).

4.4 3D 회전

3D에서는 어느 축을 중심으로 회전하는지에 따라 세 가지 기본 회전 행렬이 있다.

3D 기본 회전 행렬

x 축 회전 (roll):

$$R_x(\theta) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

y 축 회전 (pitch):

$$R_y(\theta) = \begin{pmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

z 축 회전 (yaw):

$$R_z(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

R_y 부호에 주의

y 축 회전에서 $\sin$ 부호가 다른 것에 유의하라: R_y 의 $(1, 3)$ 원소가 $+\sin \theta$, $(3, 1)$ 원소가 $-\sin \theta$. 오른손 좌표계에서 y 축 회전은 $z \rightarrow x$ 방향이기 때문이다.

5 변환의 합성

여러 변환을 차례로 적용할 때, 행렬 곱으로 합성할 수 있다.

정리 — 변환 합성

변환 $T_1, T_2, \dots, T_k$ 를 이 순서로 적용하는 것은 합성 행렬 $M = T_k \cdots T_2 T_1$ 을 곱하는 것과 같다:

$$\mathbf{p}' = T_k(\cdots T_2(T_1(\mathbf{p}))) = (T_k \cdots T_2 T_1)\mathbf{p} = M\mathbf{p}$$

주의: 행렬 곱은 오른쪽부터 적용된다 (T_1 이 먼저).

예제 1 — TRS 변환 합성

오브젝트에 순서대로 스케일(2배), 회전(z 축 45°), 이동 $(3, 1, 0)$ 을 적용하라.

$$M = T \cdot R_z \cdot S$$

$$S = \begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, R_z = \begin{pmatrix} \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} & 0 & 0 \\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, T = \begin{pmatrix} 1 & 0 & 0 & 3 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

점 $\mathbf{p} = (1, 0, 0, 1)^T$ 변환: $M\mathbf{p} = T \cdot R_z \cdot S \cdot \mathbf{p}$

$$S\mathbf{p} = (2, 0, 0, 1)^T$$

$$R_z(S\mathbf{p}) = (\sqrt{2}, \sqrt{2}, 0, 1)^T$$

$$T(R_z(S\mathbf{p})) = (\sqrt{2} + 3, \sqrt{2} + 1, 0, 1)^T$$

6 오일러 각도 (Euler Angles)

정의 — 오일러 각도

세 개의 기본 회전 각도 (α, β, γ)로 3D 방향을 표현하는 방법. 항공에서 차용한 용어:

- Roll (α): x 축 회전 (옆으로 기울기)
- Pitch (β): y 축 회전 (앞뒤로 기울기)
- Yaw (γ): z 축 회전 (좌우 방향 전환)

짐벌 락 (Gimbal Lock)

오일러 각도의 치명적 문제: 특정 배열에서 두 회전축이 겹쳐 자유도가 하나 줄어드는 현상. 예: Pitch = $\pm 90^\circ$ 일 때 Roll과 Yaw가 같은 축을 회전하게 된다.

해결책: 쿼터니언(quaternion) 사용 (강의 9에서 다룸).

7 역변환과 법선 벡터 변환

7.1 역변환

변환 M 의 역변환:

- 이동: $T^{-1} = T(-\mathbf{t})$
- 스케일: $S^{-1} = S(1/s_x, 1/s_y, 1/s_z)$
- 회전: $R^{-1} = R^T$ (직교 행렬)
- TRS 합성: $(TRS)^{-1} = S^{-1}R^{-1}T^{-1} = S^{-1}R^TT(-\mathbf{t})$

7.2 법선 벡터 변환

정리 — 법선 벡터 변환

오브젝트 표면의 법선 벡터를 변환할 때는 변환 행렬 M 이 아니라 $(M^{-1})^T$ (역행렬의 전치, 또는 전치의 역행렬)을 사용해야 한다.

$$\mathbf{n}' = (M^{-1})^T \mathbf{n}$$

균등 스케일이거나 직교 행렬(순수 회전)인 경우에는 M 으로 직접 변환해도 된다.

왜 다를까? 비균등 스케일 시 법선벡터를 M 으로 변환하면 표면에 수직이 아니게 된다. $(M^{-1})^T$ 은 이를 올바르게 보정한다.

8 뷰 행렬과 투영 행렬

8.1 뷰 행렬 (View Matrix)

카메라 위치 **eye**, 바라보는 점 **at**, 위쪽 방향 **up** 으로부터:

$$\mathbf{z}_{\text{cam}} = \frac{\mathbf{eye} - \mathbf{at}}{\|\mathbf{eye} - \mathbf{at}\|}, \quad \mathbf{x}_{\text{cam}} = \frac{\mathbf{up} \times \mathbf{z}_{\text{cam}}}{\|\mathbf{up} \times \mathbf{z}_{\text{cam}}\|}, \quad \mathbf{y}_{\text{cam}} = \mathbf{z}_{\text{cam}} \times \mathbf{x}_{\text{cam}}$$
$$V = \begin{pmatrix} x_{\text{cam},x} & x_{\text{cam},y} & x_{\text{cam},z} & -\mathbf{x}_{\text{cam}} \cdot \mathbf{eye} \\ y_{\text{cam},x} & y_{\text{cam},y} & y_{\text{cam},z} & -\mathbf{y}_{\text{cam}} \cdot \mathbf{eye} \\ z_{\text{cam},x} & z_{\text{cam},y} & z_{\text{cam},z} & -\mathbf{z}_{\text{cam}} \cdot \mathbf{eye} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

8.2 원근 투영 행렬 (Perspective Projection)

$$P = \begin{pmatrix} \frac{1}{\tan(\text{fov}/2) \cdot \text{aspect}} & 0 & 0 & 0 \\ 0 & \frac{1}{\tan(\text{fov}/2)} & 0 & 0 \\ 0 & 0 & \frac{-(f+n)}{f-n} & \frac{-2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

여기서 n 은 near 클리핑 거리, f 는 far 클리핑 거리.

9 Python 구현

실습

```
1 import numpy as np
2
3 # =====
4 # 기본변환행렬함수들
5 # =====
6
7 def translation(tx, ty, tz):
8     """이동 행렬 (4x4 동차)"""
9     T = np.eye(4)
10    T[0, 3] = tx
11    T[1, 3] = ty
12    T[2, 3] = tz
13    return T
14
15 def scale(sx, sy, sz):
16     """스케일 행렬 (4x4 동차)"""
17     return np.diag([sx, sy, sz, 1.0])
18
19 def rotation_x(theta):
20     """축x 회전형렬 """
21     c, s = np.cos(theta), np.sin(theta)
22     return np.array([
```

```

23         [1, 0, 0, 0],
24         [0, c, -s, 0],
25         [0, s, c, 0],
26         [0, 0, 0, 1]
27     ], dtype=float)
28
29 def rotation_y(theta):
30     """축y 회전행렬 """
31     c, s = np.cos(theta), np.sin(theta)
32     return np.array([
33         [ c, 0, s, 0],
34         [ 0, 1, 0, 0],
35         [-s, 0, c, 0],
36         [ 0, 0, 0, 1]
37     ], dtype=float)
38
39 def rotation_z(theta):
40     """축z 회전행렬 """
41     c, s = np.cos(theta), np.sin(theta)
42     return np.array([
43         [c, -s, 0, 0],
44         [s, c, 0, 0],
45         [0, 0, 1, 0],
46         [0, 0, 0, 1]
47     ], dtype=float)
48
49 # =====
50 # 변환적용
51 # =====
52 # 점변환 (w=1)
53 p = np.array([1.0, 0.0, 0.0, 1.0])
54
55 T = translation(3, 1, 0)
56 S = scale(2, 2, 2)
57 Rz = rotation_z(np.radians(45))
58
59 # TRS 순서: M = T * Rz * S
60 M = T @ Rz @ S
61 p_transformed = M @ p
62 print("TRS 변환결과 :", p_transformed[:3])
63
64 # 방향벡터변환 (w=0) - 이동의영향없음
65 d = np.array([1.0, 0.0, 0.0, 0.0])
66 d_transformed = T @ d
67 print("방향벡터 이동후 :", d_transformed[:3]) # 변하지않음
68
69 # =====
70 # 역변환
71 # =====
72 T_inv = translation(-3, -1, 0)

```

```

73 Rz_inv = rotation_z(np.radians(-45))
74 S_inv = scale(0.5, 0.5, 0.5)
75 M_inv = S_inv @ Rz_inv @ T_inv      #  $(TRS)^{-1} = S^{-1} R^{-1} T^{-1}$ 
76
77 print("\nM @ M_inv = I?", np.allclose(M @ M_inv, np.eye(4)))
78
79 # =====
80 # LookAt 뷰행렬
81 # =====
82 def look_at(eye, at, up):
83     z = eye - at
84     z = z / np.linalg.norm(z)
85     x = np.cross(up, z)
86     x = x / np.linalg.norm(x)
87     y = np.cross(z, x)
88     return np.array([
89         [x[0], x[1], x[2], -np.dot(x, eye)],
90         [y[0], y[1], y[2], -np.dot(y, eye)],
91         [z[0], z[1], z[2], -np.dot(z, eye)],
92         [0, 0, 0, 1]
93     ])
94
95 eye = np.array([0.0, 0.0, 5.0])
96 at = np.array([0.0, 0.0, 0.0])
97 up = np.array([0.0, 1.0, 0.0])
98 V = look_at(eye, at, up)
99 print("\뷰 행렬 V =\n", V)
100
101 # =====
102 # 법선벡터변환비균등 (스케일)
103 # =====
104 # 비균등스케일 : x=2, y=1 (방향이 y 길쭉해짐)
105 S_nonunif = scale(2, 1, 1)
106 n = np.array([1.0, 1.0, 0.0, 0.0]) # 법선벡터 (w=0)
107 n /= np.linalg.norm(n[:3])
108
109 # 잘못된변환직접 (S 적용)
110 n_wrong = S_nonunif @ n
111 print("\잘못된 법선:", n_wrong[:3])
112
113 # 올바른변환역전치 ()
114 N_matrix = np.linalg.inv(S_nonunif).T
115 n_correct = N_matrix @ n
116 n_correct = n_correct / np.linalg.norm(n_correct[:3])
117 print("올바른 법선:", n_correct[:3])

```

10 연습 문제

연습 문제

1. 다음 변환을 4×4 동차 행렬로 작성하고, 점 $(1, 2, 0)$ 에 적용하라: (a) x 방향 5 이동, (b) z 축 90° 회전, (c) 균등 스케일 3배.
2. 변환 순서 “먼저 z 축 30° 회전, 그 다음 $(2, 0, 0)$ 이동”과 “먼저 $(2, 0, 0)$ 이동, 그 다음 z 축 30° 회전”의 합성 행렬을 각각 구하고 결과가 다름을 보여라.
3. 오브젝트를 중심이 (c_x, c_y, c_z) 인 점 기준으로 θ 회전시키는 합성 변환 행렬 M 을 유도하라. (힌트: 이동 $\rightarrow$ 회전 $\rightarrow$ 역이동)
4. 직교 행렬(순수 회전) R 에 대해 $R^{-1} = R^T$ 임을 행렬식 관점에서 증명하라.
5. Python으로 오브젝트를 1초에 45° 씩 z 축 회전하는 애니메이션을 시뮬레이션하라. 10 타임스텝 동안 점 $(1, 0, 0)$ 의 위치를 출력하라.

11 요약

핵심 정리

1. 어파인 변환 = 선형 변환 + 이동; 이동을 행렬 곱으로 표현하려면 동차 좌표계 필요.
2. 동차 좌표: 점 $\rightarrow w = 1$, 방향 벡터 $\rightarrow w = 0$.
3. 기본 변환: 이동 T , 스케일 S , 회전 R_x, R_y, R_z .
4. 합성: $M = T \cdot R \cdot S$ (역순으로 적용 — S 먼저).
5. 역변환: $T^{-1} = T(-t)$, $S^{-1} = S(1/s)$, $R^{-1} = R^T$.
6. 법선 변환: 비균등 스케일 시 $(M^{-1})^T$ 사용.
7. 오일러 각도의 짐벌 락 문제는 쿼터니언으로 해결.